

Acassist — Team Build Sheet

Build Over Nights · Education Crisis track · Build day: Saturday, Oct 3, 2026 · ~12-hour single block

For the team building it — the interface, the scope, and the lines we don't cross. Not a pitch doc.

Architecture recap (context)

- **Tauri desktop app.** UI is raw HTML/CSS/vanilla JS in the webview; the scoring/hashing lives in the native Rust core (Tauri command bridge).
- **Modules are JSON, rendered by fixed webview JS** — a renderer, not a transformer/compiler. Same JSON can be shown as a lesson, quiz, or flashcards.
- **Scoring is device-neutral hashing:** SHA256(question id : normalized answer : salt), no device id — same hash on every device, distributes any way (file, USB, network) and scores offline. This is the novel core.

The JSON contract (spec — the interface all three lanes build against)

This is the *shape* teacher authoring, the Rust scoring core, and the student UI all agree on. Specification, not code — aligning on it before the build lets the lanes work in parallel from hour zero. A module is a plain .json file (never a custom format or DSL).

```
```jsonc
{
 "schema_version": "1.0",
 "module": {
 "id": "mod_photosynthesis_01",
 "type": "quiz", // "quiz" | "lesson" — renderer switches on this
 "title": "Photosynthesis Basics",
 "grade_level": "elementary" // metadata only
 },
 "lesson": { // render-only content
 "blocks": [
 { "kind": "heading", "text": "What is Photosynthesis?" },
 { "kind": "paragraph", "text": "Plants make food from sunlight..." }
]
 },
 "quiz": {
 "hash_algo": "SHA256",
 "normalization": "lowercase|trim|strip_punctuation", // the CONTRACT both sides obey
 "questions": [
 {
 "id": "q1", // stable — part of the hash input
 "kind": "multiple_choice", // "multiple_choice" | "identification" | "true_false"
 "prompt": "Which gas do plants take in?",
 "options": ["Oxygen", "Carbon Dioxide", "Nitrogen", "Hydrogen"],
 "salt": "a3f9c2...", // per-question random salt, shipped in the file
 }
]
 }
}
```

```

"answer_hash": "9b74c9...", // SHA256(q.id : normalized_answer : salt)
"points": 1
}
]
}
}
}
...

```

## Load-bearing rules

- **No plaintext answers, ever.** The correct answer in the clear exists only on the teacher side at seal time. The shipped module carries `answer_hash + salt` — never the answer itself. This is what makes "no answer key on disk" true.
- **id and salt are frozen once sealed** — both are hash inputs. Question ids are immutable identifiers, not display order; changing either requires recomputing the hash.
- **normalization is a contract, not a comment.** Rust normalizes the correct answer before hashing (seal); JS normalizes the student's input *identically* before the core checks it. Any mismatch = "correct answer marked wrong" bugs. One source of truth, mirrored exactly on both sides.
- **Hash the answer text, not the option index** — keeps one code path with identification questions. MC is inherently low-entropy; that's the honest "tamper-resistant, not unbreakable" story, not something fixed here.

## Seal vs. check flow

JS collects raw input → normalizes per the contract → hands (`id`, `normalized_answer`) to the Rust core → core computes `SHA256(id : normalized_answer : salt)` and compares to `answer_hash` → returns correct/incorrect → JS tallies points. The core only ever *recomputes* from what the student typed; it never reads a plaintext answer, because there isn't one.

The flashcard render (a SHOULD) reuses this same JSON — front = prompt, back = the answer option or a teacher-provided explanation — so no separate flashcard shape is needed.

## Build scope

Everything competes with the offline scoring core, and the core wins every time.

**MUST — the sacred flow (protect above all).** If only this works, the demo still wins.

1. Rust-core offline device-neutral scoring — `SHA256(id : normalized answer : salt)`, no device id; normalize identically in Rust (seal) and JS (input). Hit the ■ checkpoint here *first*.
2. JSON module schema + loader — load from a local JSON file (simulates "pushed"; do **not** build real distribution).
3. Quiz-take UI — take, submit, see score, offline.
4. Lesson render view — render-only text/markdown.
5. One AI-generated (Groq) lesson/quiz module, teacher-side, online — with a **pre-generated fallback module** so a failed live call never breaks the demo.

**SHOULD — only after the ■ checkpoint holds, with hours to spare.**

- Deterministic offline flashcard render — "same JSON, another view," pure transform, no AI. First thing to cut if the core is shaky.

- Deterministic prompting scaffolds for the teacher (templates so non-techy teachers down to elementary aren't facing a blank box).

**ROADMAP — pitch it, do NOT build at the event.**

- Online AI enhancement of study aids (Groq-boosted flashcards/hints when connected).
- Bind-at-download / PIN-bound hardening (two flavors).
- Real teacher→student network distribution, accounts, three roles.
- Rich lesson authoring, media, TTS/read-aloud, analytics.

**DEAD — never raise again.** Agora (any use), a custom DSL, a crypto/hash arms race, an embedded runtime AI on the student side (offline), a standalone AI assistant.

## **The line we hold**

Resist your own cleverness. No DSL, no crypto arms race, no accounts. The visible, working, understandable offline-scoring flow is what wins — not the elegant machinery underneath. Core first, always.